



ARQUITECTURA Y SISTEMAS OPERATIVOS

CLASE 4

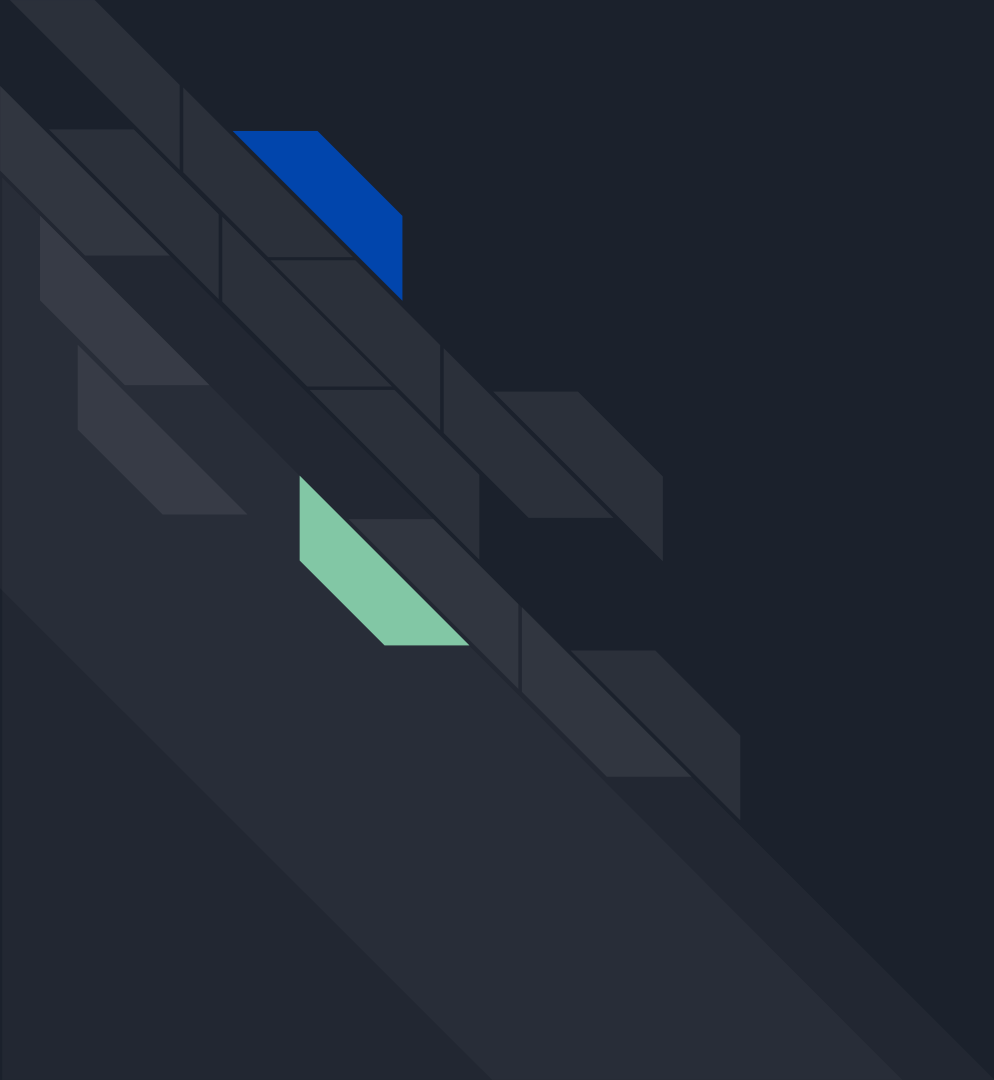
Hilos (subprocesos) y Concurrency

Temas de la clase

En la clase de hoy vamos a ver:

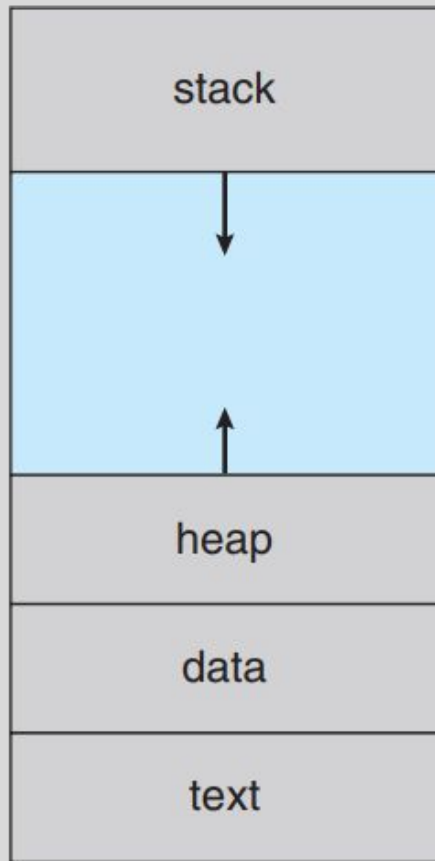
- Qué es un hilo
- Proceso vs hilo
- Beneficios del multihilo
- Concurrencia y paralelismo
- Modelos de hilos
- Librerías de hilos





REPASO

max



0



Procesos

Un proceso es un programa en ejecución.

Cuando un programa se ejecuta, el SO debe crear una estructura de control que le permita administrarlo:

- su espacio de memoria
- su estado actual
- el contexto de CPU
- y los recursos que utiliza

El proceso es la entidad administrada por el SO.

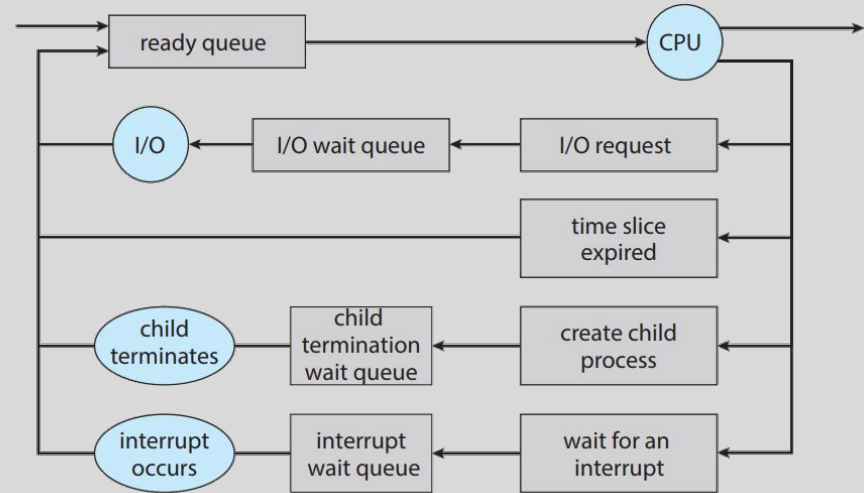
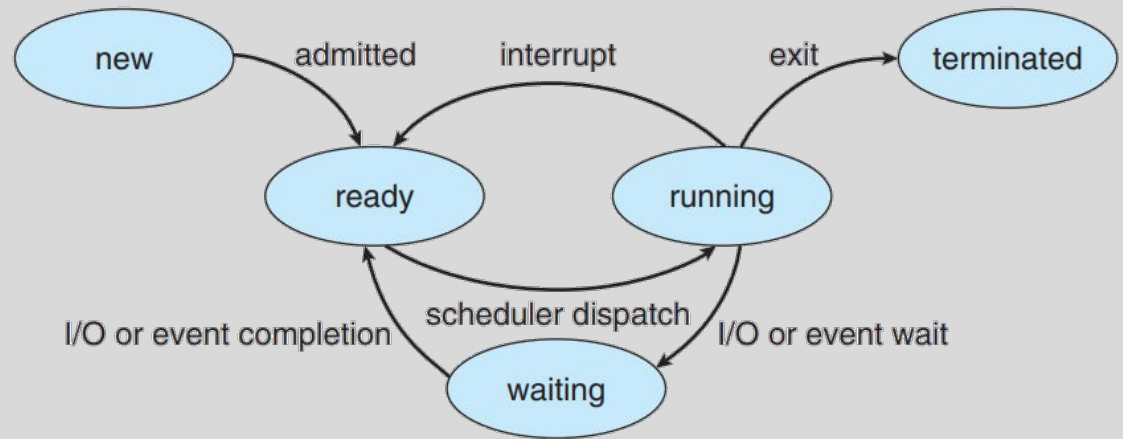
Es decir, el SO no ejecuta “código suelto”, ejecuta procesos, y para hacerlo debe guardar su contexto y sus recursos.

Estados y Ciclo de vida

Estados: nuevo, ejecutando, esperando, listo o finalizado.

En ejecución, el proceso puede salir de la CPU por:

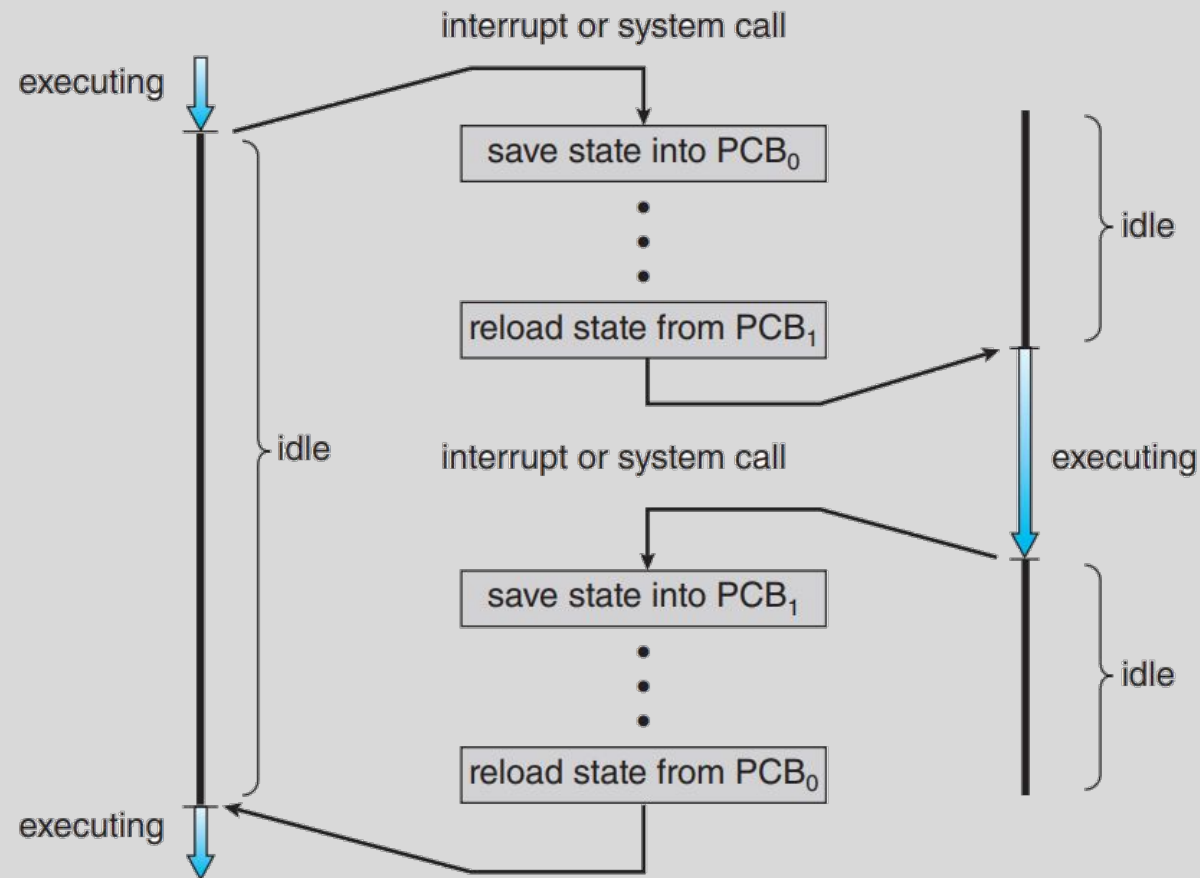
1. **Bloqueo por E/S (I/O):** El proceso se **detiene voluntariamente** porque necesita datos externos (disco, red).
2. **Creación de Hijo (Fork):** El proceso crea un nuevo proceso y espera hasta que el hijo termine su tarea.
3. **Expulsión (Preemption):** El SO le quita el control del CPU **por fuerza**.
4. **Finalización (Exit):** El proceso termina con éxito todas sus tareas.



process P_0

operating system

process P_1



Cambio de Contexto

Mecanismo para alternar entre procesos.

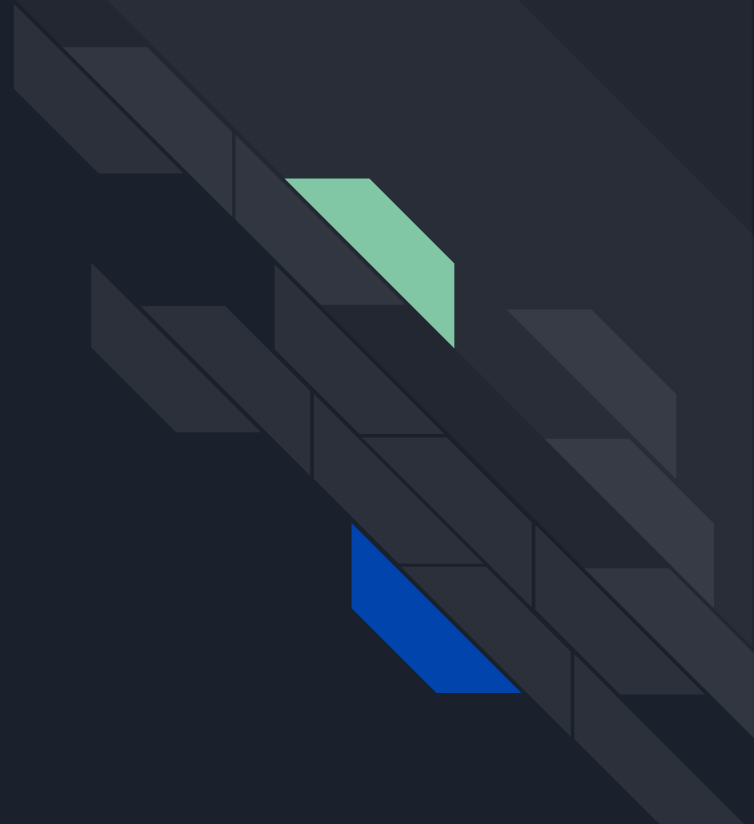
- **Guarda el estado** (contexto) actual del proceso saliente y **restaura el estado** del siguiente proceso a ejecutarse.
- Toda la información técnica se almacena en el PCB del proceso que se suspende.
- Este intercambio es **tiempo perdido** para el usuario, ya que la CPU no realiza cálculo útil.
- Su **velocidad** real depende del hardware subyacente para manejar estas transiciones.



BIBLIOGRAFIA

- Operating System Concepts. By Abraham, Silberschatz.
 - Capítulo IV
- Modern Operating Systems. By Andrew S. Tanenbaum.
 - Capítulo II

THREADS (HILOS) Y CONCURRENCIA





El límite del proceso tradicional

En el modelo más simple, pensamos a un proceso como una única secuencia de instrucciones ejecutándose.

Y si bien ese modelo sirve para entender muchas ideas básicas, empieza a quedar corto cuando una aplicación necesita realizar varias actividades relacionadas entre sí.

Por ejemplo, una aplicación puede necesitar al mismo tiempo:

- Atender interacción del usuario
- Esperar datos desde disco o red
- Ejecutar tareas periódicas
- Procesar información en segundo plano
- Responder múltiples pedidos externos



El límite del proceso tradicional

Resolver esto creando un proceso separado para cada actividad, generaría varios problemas:

- Mayor costo de creación y destrucción
- Más costo de cambio de contexto
- Dificultad para compartir datos
- Necesidad de mecanismos explícitos de comunicación entre procesos

Entonces aparece una necesidad más sofisticada que “crear más procesos”, cuál es la idea?:

- **tener múltiples flujos de ejecución dentro del mismo proceso.**



Qué es un hilo

Un **hilo** o *thread* es la **unidad de ejecución más pequeña dentro de un proceso**. El **proceso** es la entidad que **agrupa recursos**, el **hilo** es la entidad que **avanza ejecutando instrucciones**.

- El proceso define el entorno de ejecución
- El hilo define la secuencia de instrucciones que se está ejecutando.

Un proceso puede tener:

- Un solo hilo de ejecución, o
- múltiples hilos que avanzan dentro del mismo espacio de direcciones.

Idea central: un proceso no necesariamente equivale a “una sola actividad”.

Un mismo proceso puede contener varias actividades ejecutándose de manera concurrente.

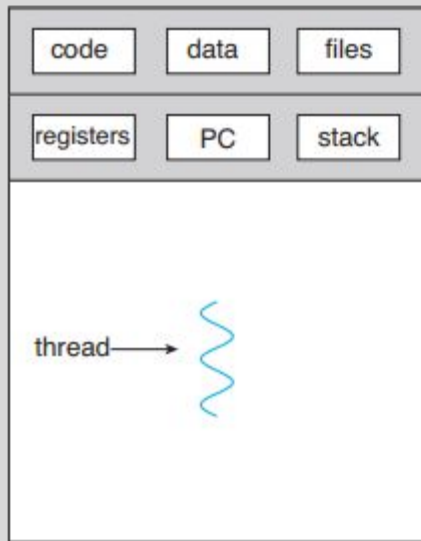
Qué tiene un hilo propio

Varios hilos pueden pertenecer al mismo proceso, por eso necesitan conservar su propio estado de ejecución:

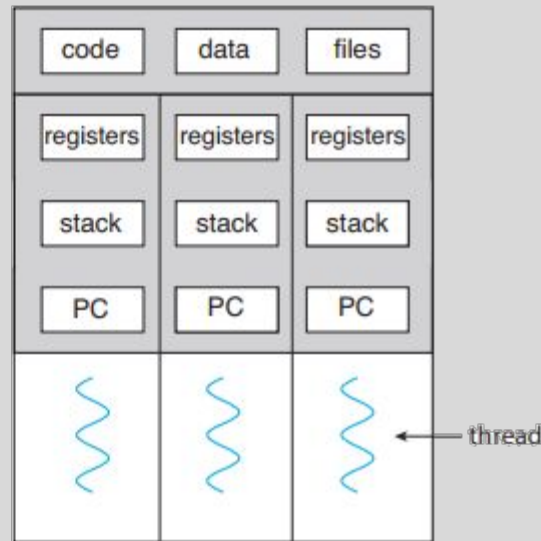
- Identificador de hilo (ID)
- Contador de programa (PC)
- Conjunto de registros
- Y una **stack** propia.

En el **stack** se almacenan:

- Parámetros de funciones
- Variables locales
- Direcciones de retorno
- Y datos temporales de ejecución



single-threaded process

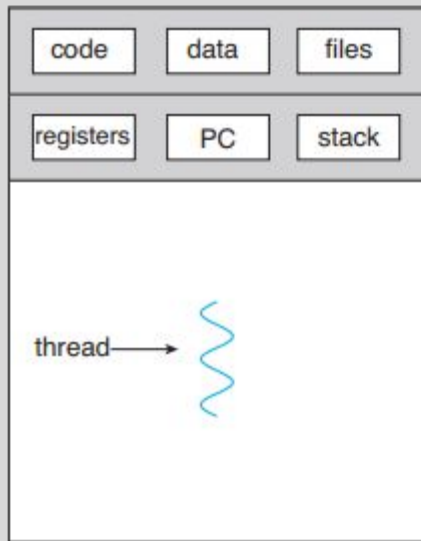


multithreaded process

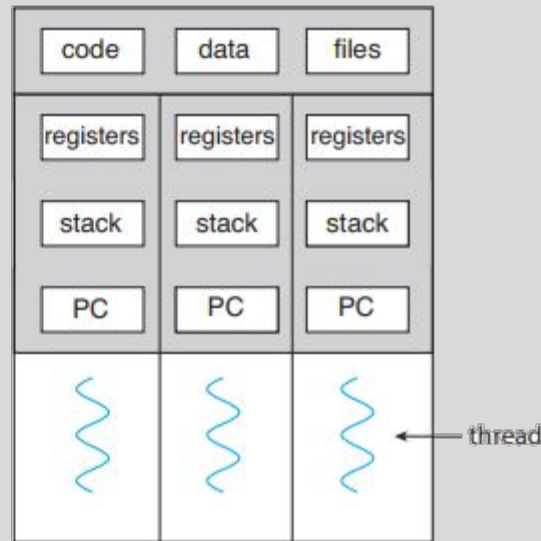
Qué tiene un hilo propio

Entender que dos hilos del mismo proceso pueden estar ejecutando funciones distintas, con variables locales distintas, sin pisarse entre sí.

Aunque compartan memoria general del proceso, cada hilo necesita su propio contexto para poder ser pausado y reanudado correctamente.



single-threaded process



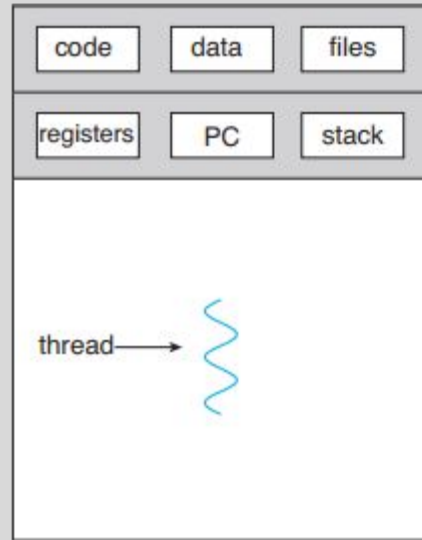
multithreaded process

Qué comparten los hilos de un proceso

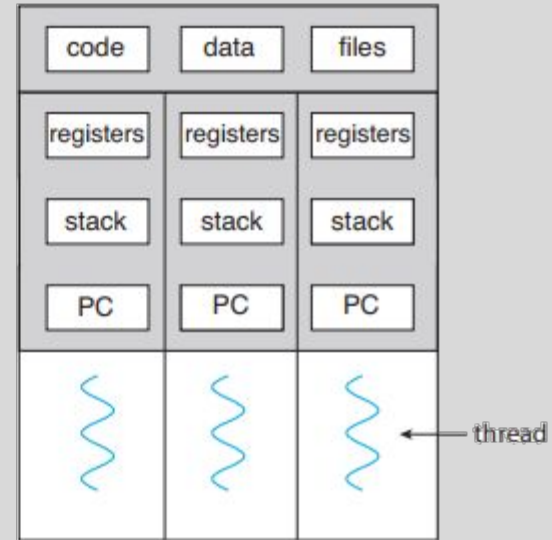
Comparten el entorno general del proceso al que pertenecen.

Esto es:

- La sección **text**, el código ejecutable.
- La sección **data**, variables globales.
- El *heap*, la memoria dinámica.
- Archivos abiertos.
- Y otros recursos del proceso.



single-threaded process



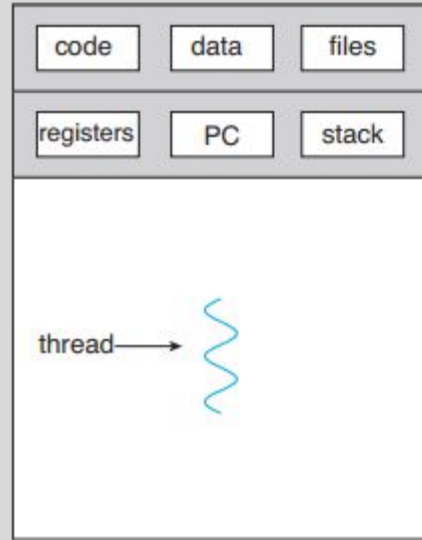
multithreaded process

Qué comparten los hilos de un proceso

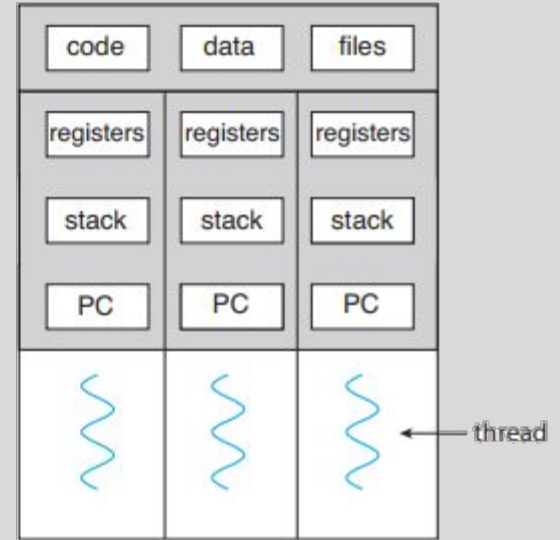
A diferencia de procesos separados, los hilos de un mismo proceso nacen dentro del mismo **espacio de direcciones**.

Esto permite que varias actividades colaboren directamente sobre los mismos datos.

Pero tiene su costo, si varios hilos acceden al mismo dato al mismo tiempo, aparecerán problemas de coordinación y sincronización.



single-threaded process

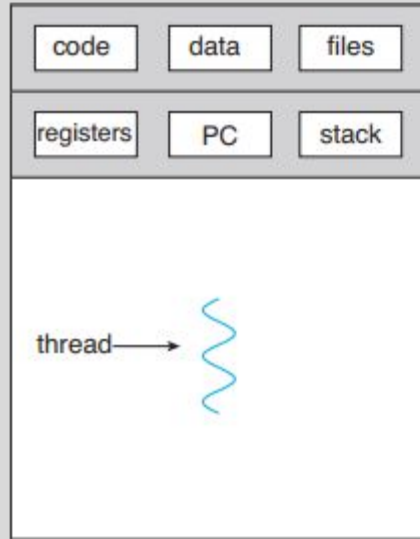


multithreaded process

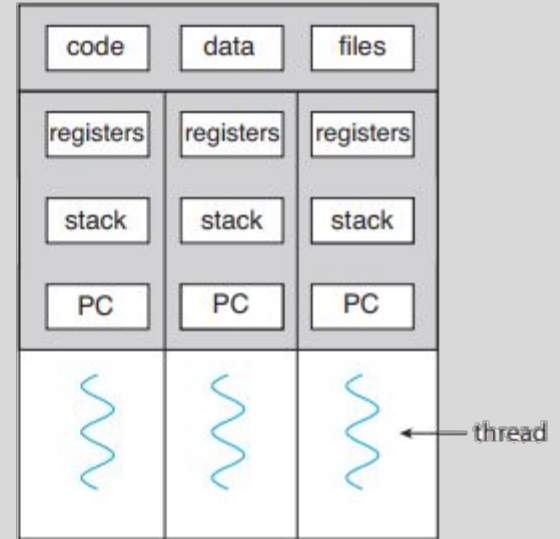
Proceso monohilo vs proceso multihilo

Proceso monohilo: una sola secuencia de ejecución.

- Hay un hilo avanzando por sus instrucciones. Si ese hilo se bloquea o demora, toda la ejecución del proceso queda detenida.



single-threaded process



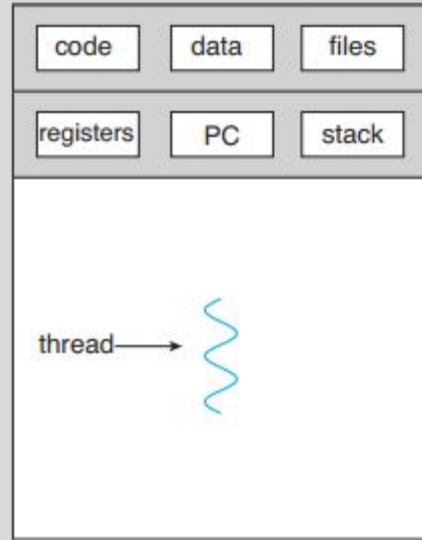
multithreaded process

Proceso monohilo vs proceso multihilo

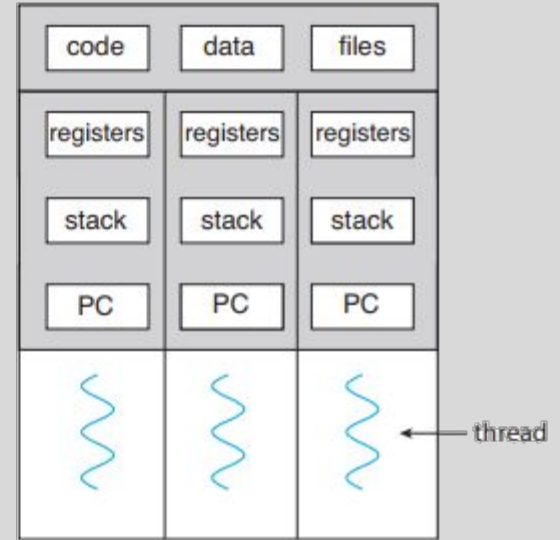
Proceso multihilo: varios hilos de ejecución dentro del mismo proceso.

- Cada hilo tiene su propia stack y contexto.
- Todos comparten código, datos y recursos del proceso.

Según el hardware disponible, esto permite que distintas partes de una misma aplicación avancen en paralelo o de manera concurrente.



single-threaded process

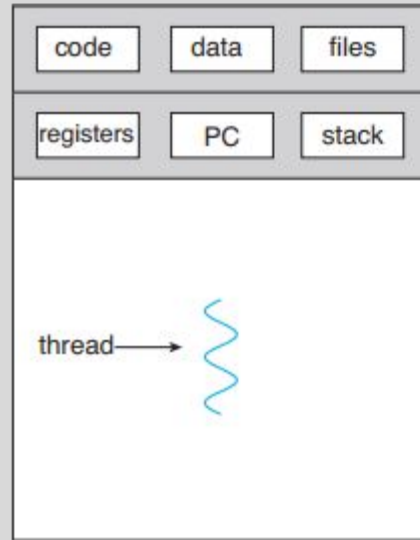


multithreaded process

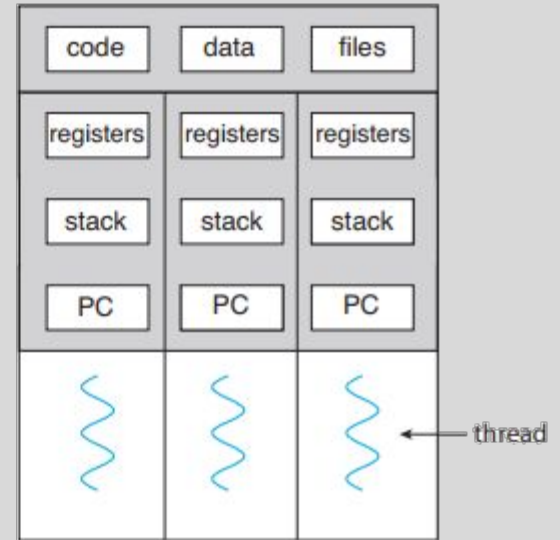
Proceso monohilo vs proceso multihilo

Entender:

- En un proceso monohilo, una sola actividad representa toda la ejecución.
- En un proceso multihilo, varias actividades relacionadas coexisten dentro de la misma aplicación.



single-threaded process



multithreaded process



Por qué aparecen los hilos

Porque muchas aplicaciones reales, en general no hacen una única tarea de principio a fin, sino que combinan múltiples actividades relacionadas.

Ejemplo:

- Interfaz gráfica espera eventos del usuario.
- Módulo de red espera respuestas del exterior.
- Otra parte procesa datos.
- Y otra realiza tareas periódicas o en segundo plano.

Todo eso dentro de una única secuencia de ejecución, volvería a la aplicación menos flexible:

- Una tarea lenta podría frenar a las demás.
- Una espera de E/S podría bloquear toda la lógica.
- Y la estructura del programa sería más difícil de organizar.



Por qué aparecen los hilos

Los hilos permiten separar actividades en flujos de ejecución distintos, conservando una única estructura de proceso y un único espacio de memoria compartido.

Además de ser herramienta de rendimiento, también son una herramienta de organización del programa.

Los hilos no solo sirven para “ir más rápido”. También sirven para:

- Estructurar mejor el software
- Desacoplar actividades
- Evitar que una parte detenga a toda la aplicación

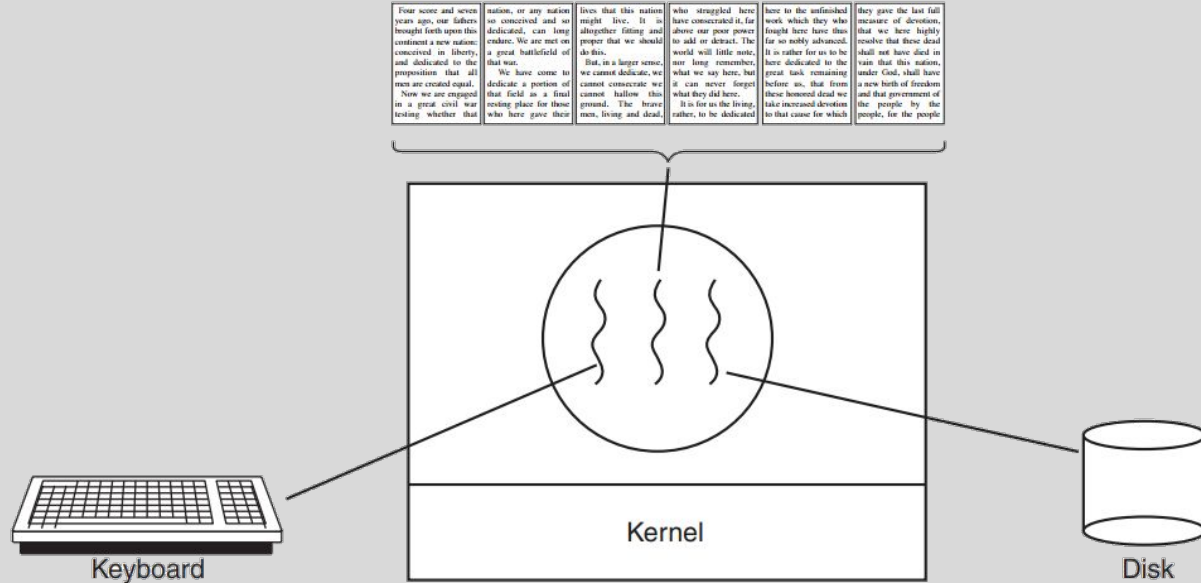
Ejemplo I

Un procesador de texto simple que realiza 3 tareas en simultáneo:

1. Atender la E/S que ingresa el usuario por teclado.
2. Ejecutar la función de auto-corrector.
3. Realizar un *backup* del documento periódicamente.

Estas actividades no son independientes: **todas trabajan sobre el mismo documento abierto.**

Para resolver esto con procesos separados, habría que compartir el documento y mantenerlo consistente, con mecanismos de comunicación y coordinación adicionales.

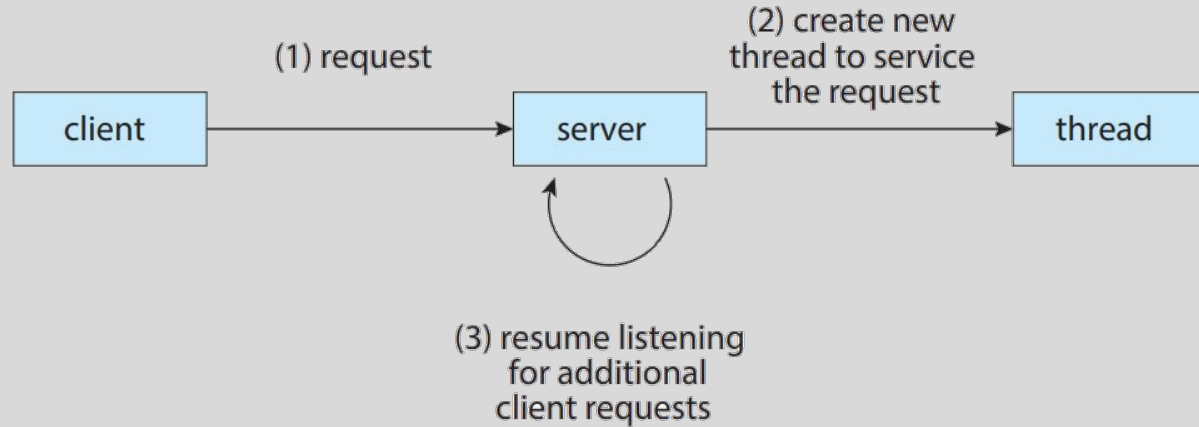


Ejemplo II

Un servidor Web:

- Espera nuevas solicitudes
- Atiende clientes que ya hicieron pedidos
- Procesa esos pedidos
- Accede a archivos, bases de datos o servicios externos
- Y debe seguir disponible para futuras conexiones

Si cada vez que llega una solicitud el servidor crea un proceso nuevo, el costo de creación y administración sería alto, más aún si recibe muchas solicitudes simultáneamente.

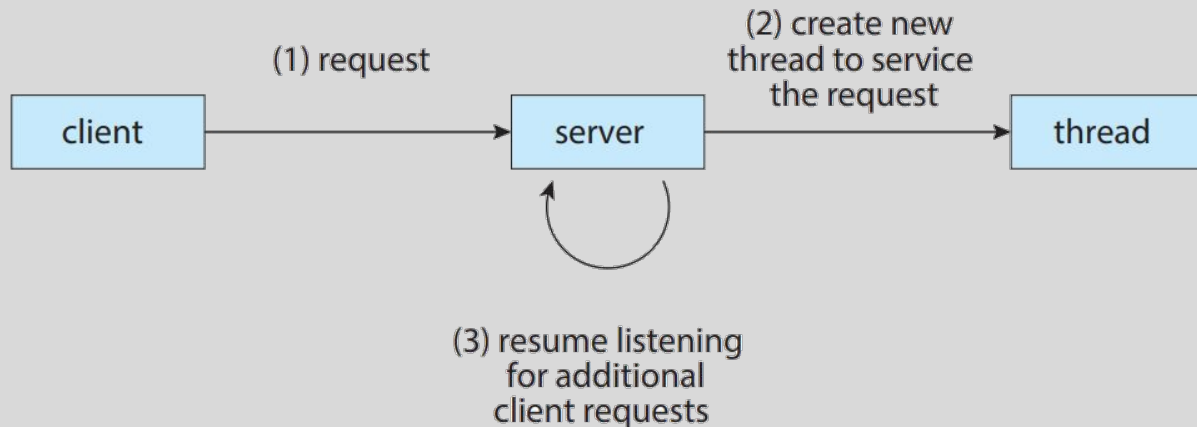


Ejemplo II

Un proceso con múltiples hilos:

- Un hilo puede aceptar nuevas conexiones
- Mientras otros hilos procesan solicitudes ya recibidas.

Esto mejora la organización y el rendimiento. Crear y cambiar entre hilos es menos costoso que hacerlo entre procesos.





Beneficios 1

Capacidad de respuesta

En una aplicación con varios hilos, una parte del programa puede continuar aunque otra parte esté bloqueada o demore más tiempo. Por ejemplo:

- Un hilo espera datos desde la red, y otro sigue atendiendo eventos del usuario.
- Un hilo guarda información en disco, y otro mantiene actualizada la interfaz.

Desde el punto de vista del usuario, esto hace que el programa se perciba más fluido y más reactivo.



Beneficios 2

Uso compartido de recursos

En procesos separados, compartir datos no es automático. El programador debe construir mecanismos adicionales: memoria compartida, sockets, colas de mensajes, etc.

En cambio, los hilos de un mismo proceso comparten:

- Espacio de direcciones
- Código
- Datos globales
- El heap
- Y otros recursos del proceso

Acá la cooperación entre distintas partes de una misma aplicación es mucho más directa.



Beneficios 2

Uso compartido de recursos

Desventaja:

- Si varios hilos acceden y modifican la misma información al mismo tiempo, pueden aparecer inconsistencias.

Por eso, compartir recursos vuelve más natural el diseño de ciertas aplicaciones, pero también introduce la necesidad de **coordinación** (sincronización).



Beneficios 3

Economía

Crear un proceso implica reservar y preparar una estructura bastante más pesada:

- Espacio de direcciones propio
- Estructuras de control del SO
- Recursos asociados
- Más trabajo administrativo para el sistema

En cambio, crear un hilo dentro de un proceso existente es más barato. No necesita un nuevo espacio de memoria ni duplicar todos los recursos.



Beneficios 3

Economía

También:

- Destruir un hilo es menos costoso que destruir un proceso
- Cambiar de un hilo a otro tiene menos sobrecarga que cambiar procesos



Beneficios 4

Escalabilidad

En sistemas con múltiples núcleos de procesamiento, una aplicación multihilo puede aprovechar mejor el hardware disponible.

En una aplicación con un único hilo, su ejecución queda limitada a **una sola secuencia de instrucciones**. Aunque el sistema tenga varios núcleos disponibles.



Beneficios 4

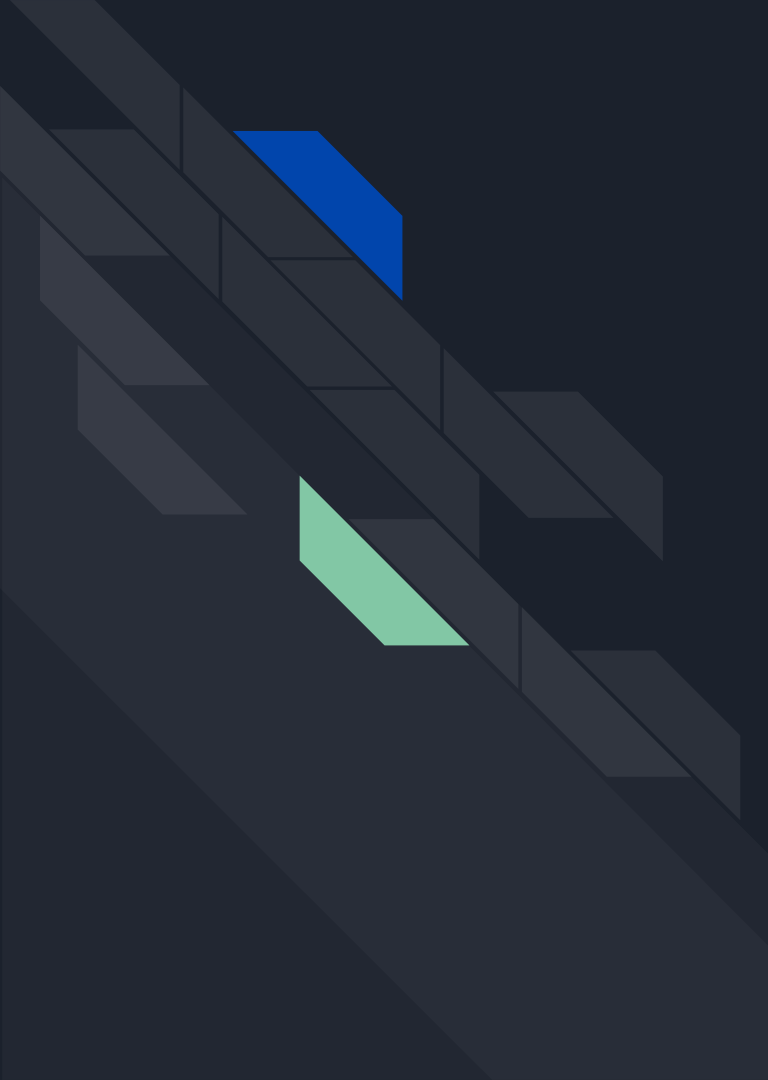
Escalabilidad

En cambio, en una aplicación con varios hilos, el SO puede planificarlos en distintos núcleos. Eso genera que distintas partes del proceso avancen **realmente al mismo tiempo**.

Aunque no garantiza automáticamente mejor rendimiento, depende de:

- Cómo esté diseñado el programa
- Si las tareas pueden dividirse
- Si compiten por los mismos datos
- Del costo de coordinación entre hilos

Pero conceptualmente, múltiples hilos permiten que una aplicación escale mejor cuando el hardware ofrece varios núcleos.



PROGRAMACIÓN EN SISTEMAS MULTINÚCLEO



De un núcleo a varios núcleos

Para entender mejor el impacto del multihilo, también hay que mirar el hardware.

Por lo dicho anteriormente, no es lo mismo ejecutar varios hilos en un sistema con un solo núcleo, que en un sistema con varios núcleos de procesamiento.

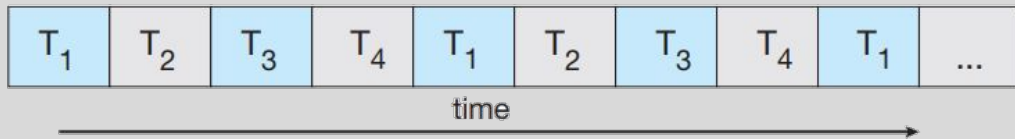
- En ambos casos puede existir multihilo, pero el resultado práctico no es idéntico.

En un sistema de un solo núcleo, todos los hilos deben turnarse para usar el procesador. En un sistema multinúcleo, distintos hilos pueden ser asignados a distintos núcleos, lo que permite simultaneidad real en ciertos casos.

Por eso, a partir de este punto conviene distinguir dos ideas relacionadas pero no equivalentes:

- Concurrencia
- Paralelismo

single core



Concurrent execution on a single-core system.

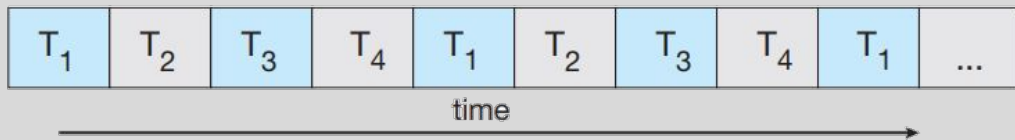
Single-core: conurrencia por intercalado

En sistemas *single-core*, el procesador solo puede ejecutar un hilo a la vez.

Aunque una aplicación tenga varios hilos, **no avanzan simultáneamente** en sentido físico. El SO alterna entre ellos, asignando a cada uno pequeños intervalos de CPU.

Así, en un único núcleo podemos tener **conurrencia**, pero no **paralelismo real**.

single core



Concurrent execution on a single-core system.



Parallel execution on a multicore system.

Multicore: posibilidad de paralelismo real

En sistemas *multicore*, el hardware ofrece más de un núcleo de ejecución.

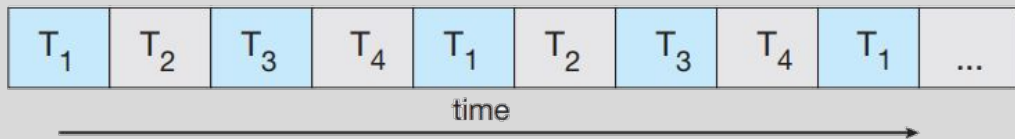
Eso permite que distintos hilos puedan ser ejecutados en núcleos distintos al mismo tiempo.

Acá sí existe la posibilidad de **simultaneidad real** entre tareas.

Por ejemplo:

- Un hilo puede estar procesando datos en un núcleo.
- Otro atiende una solicitud en otro núcleo.
- Y un tercero realiza otra tarea en un tercer núcleo.

single core



Concurrent execution on a single-core system.



Parallel execution on a multicore system.

Multicore: posibilidad de paralelismo real

Todo esto depende de varios factores:

- Que existan hilos listos para ejecutar.
- Que el trabajo pueda dividirse.
- Que no haya dependencias que obliguen a esperar.
- Y que el SO los planifique adecuadamente.

Desde el punto de vista conceptual, *multicore* hace posible que múltiples hilos no solo organicen mejor la aplicación, sino que también habilita paralelismo real.



Concurrencia y paralelismo no son lo mismo

Aunque suelen usarse juntas, no significan lo mismo.

- Un sistema **concurrente** es aquel en el que múltiples tareas pueden avanzar y progresar de manera coordinada, aunque no necesariamente al mismo tiempo.
- Un sistema **paralelo** es aquel en el que múltiples tareas se ejecutan simultáneamente.

Entonces:

- **Concurrencia** se relaciona con la estructura del trabajo y su avance.
- **Paralelismo** se relaciona con la ejecución simultánea efectiva.

Esto nos lleva a mencionar 2 tipos de paralelismo:

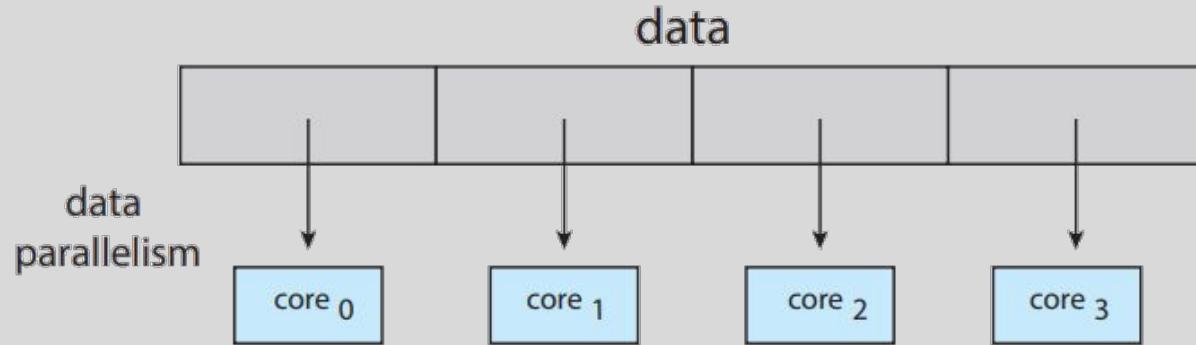
- De datos
- De tareas

Paralelismo de datos

La idea es dividir un conjunto de datos en partes y aplicar sobre cada una la misma operación.

Por ejemplo:

- Procesar distintos fragmentos de una imagen
- Sumar o transformar elementos de un arreglo grande
- Recorrer distintas porciones de una matriz



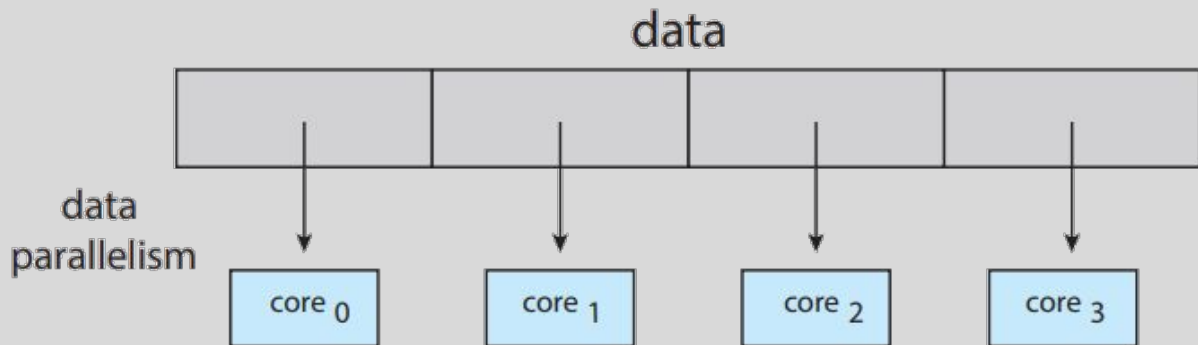
Paralelismo de datos

En este caso:

- El trabajo es esencialmente el mismo.
- Los datos se reparten entre distintos hilos o núcleos.

Esto funciona bien cuando:

- Los datos pueden dividirse.
- La operación sobre cada parte es similar.
- No existen dependencias entre las partes.



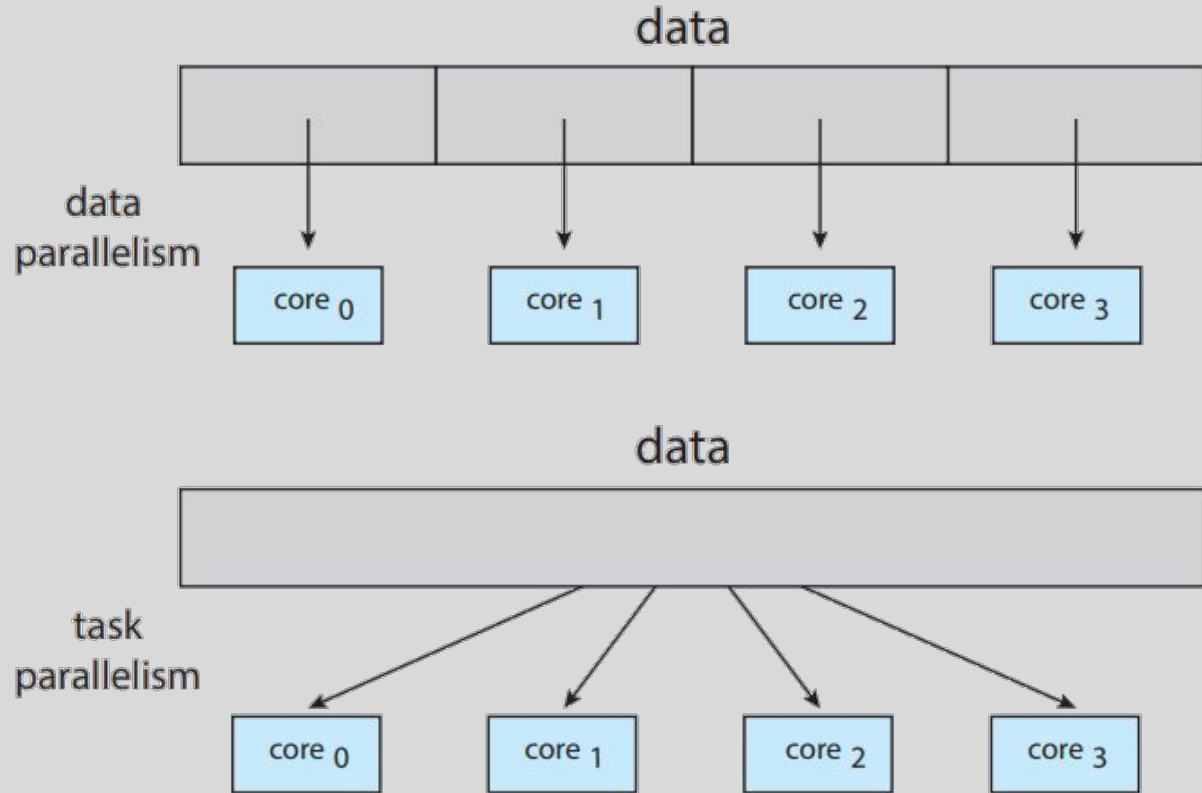
Paralelismo de tareas

En este caso, las actividades son distintas.

Cada hilo realiza una tarea específica:

- Una puede leer datos
- Otra procesarlos
- Otra guardarlos
- Y otra atiende eventos del usuario.

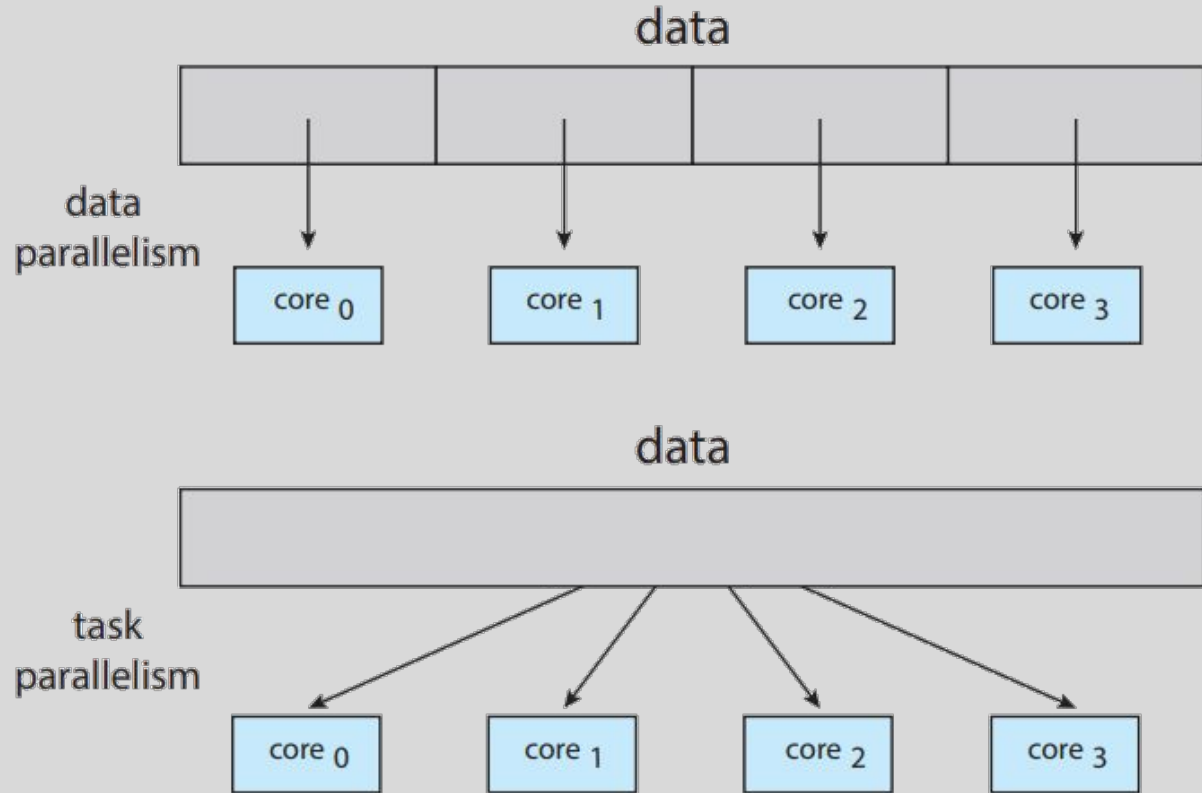
Acá, no necesariamente todos los hilos ejecutan lo mismo. Lo importante es que el trabajo total del sistema se divide en funciones diferentes.



Paralelismo de datos vs tareas

Idea:

- **Paralelismo de datos** = misma operación, distintos datos.
- **Paralelismo de tareas** = distintas operaciones





Programar con varios hilos no es gratis

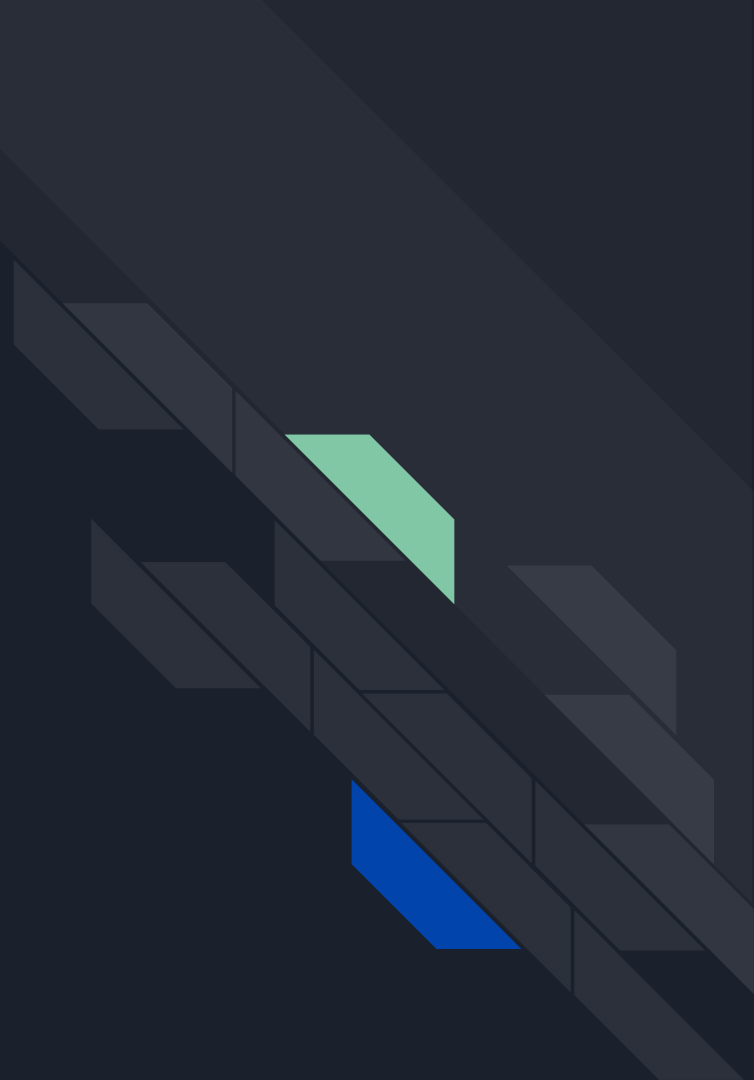
Aunque múltiples hilos ofrece ventajas importantes, programarlos también introduce dificultades.

Para que una aplicación aproveche bien varios flujos de ejecución, hay que resolver varios problemas de diseño:

- Identificar qué partes del trabajo pueden separarse.
- Lograr que la división tenga sentido.
- Equilibrar la carga entre tareas.
- Coordinar accesos a los datos.
- Probar correctamente comportamientos que pueden variar de una ejecución a otra.

Múltiples hilos aportan potencia y flexibilidad, pero a cambio agrega complejidad conceptual y técnica.

MODELOS MULTITHILOS



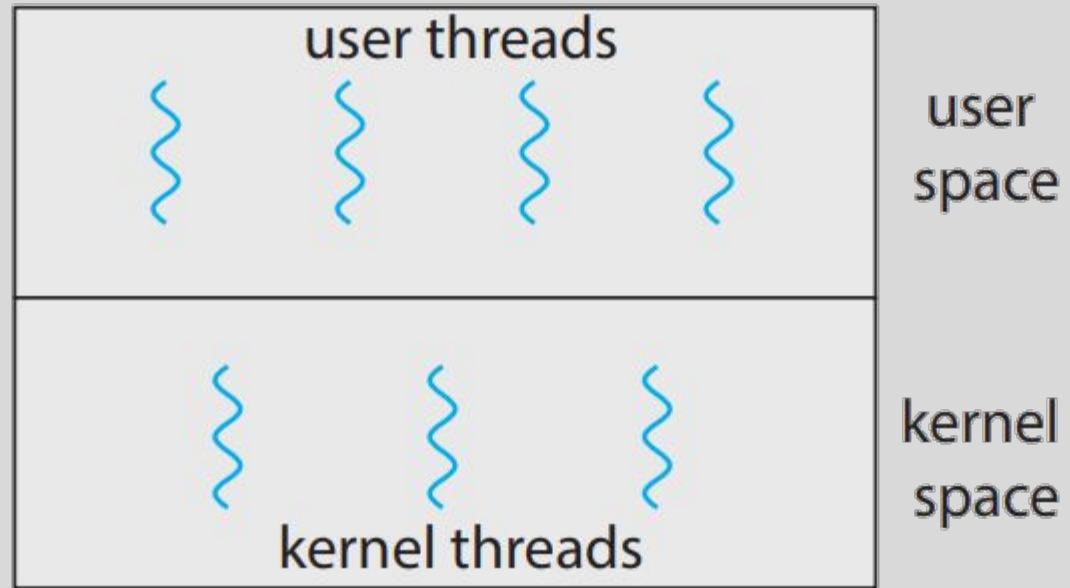
Hilos de usuario e hilos de kernel

En términos generales, podemos distinguir:

- **Hilos de usuario**, gestionados por una librería en espacio de usuario.
- **Hilos de kernel**, gestionados directamente por el SO.

Los hilos de usuario suelen ser más livianos porque muchas operaciones se resuelven sin entrar al kernel.

Los hilos de kernel son visibles para el SO, y puede planificarlos directamente sobre los procesadores.



User and kernel threads.

Hilos de usuario e hilos de kernel

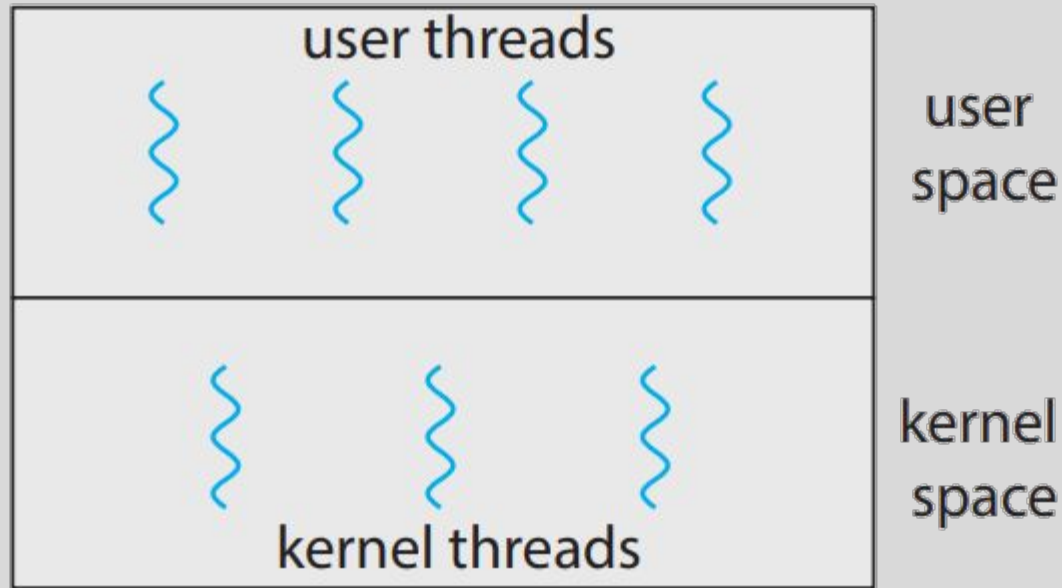
Idea:

- Una cosa es el hilo “visto por la aplicación”
- Y otra es el hilo “visible para el SO”.

Esto obliga a definir una relación:

- Cómo se conectan los hilos creados por la aplicación con las entidades que realmente planifica el SO.

A partir de ahí aparecen distintos **modelos de mapeo** entre hilos de usuario y hilos de kernel.



User and kernel threads.

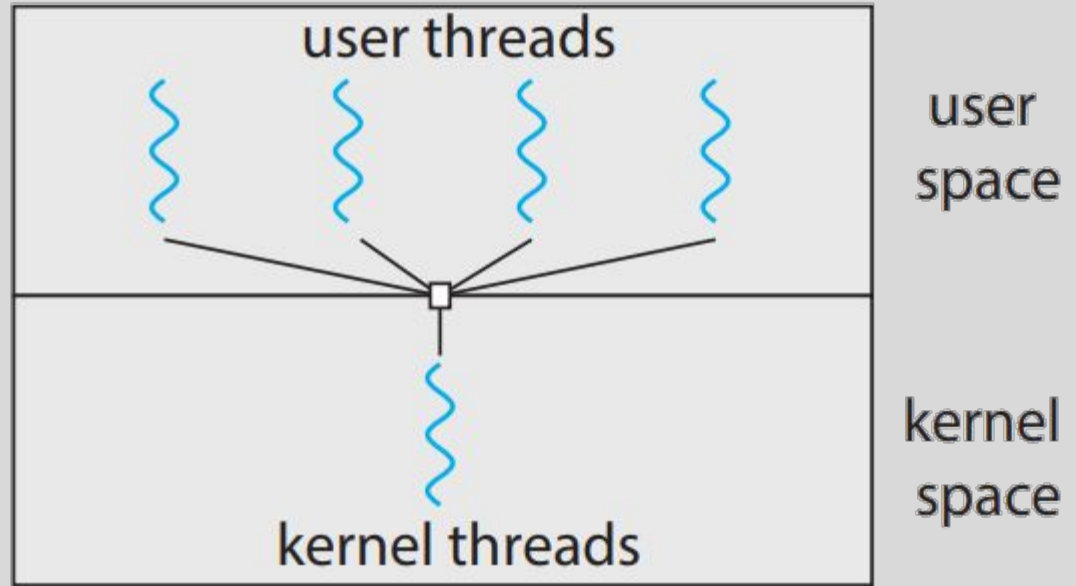
Modelo muchos a uno

Varios hilos de usuario se asocian a un único hilo de kernel.

La aplicación puede crear múltiples hilos desde su punto de vista, pero para el SO todos terminan dependiendo de una sola entidad de ejecución a nivel de kernel.

Ventaja:

- La gestión de hilos en espacio de usuario puede ser rápida y liviana.



Many-to-one model.

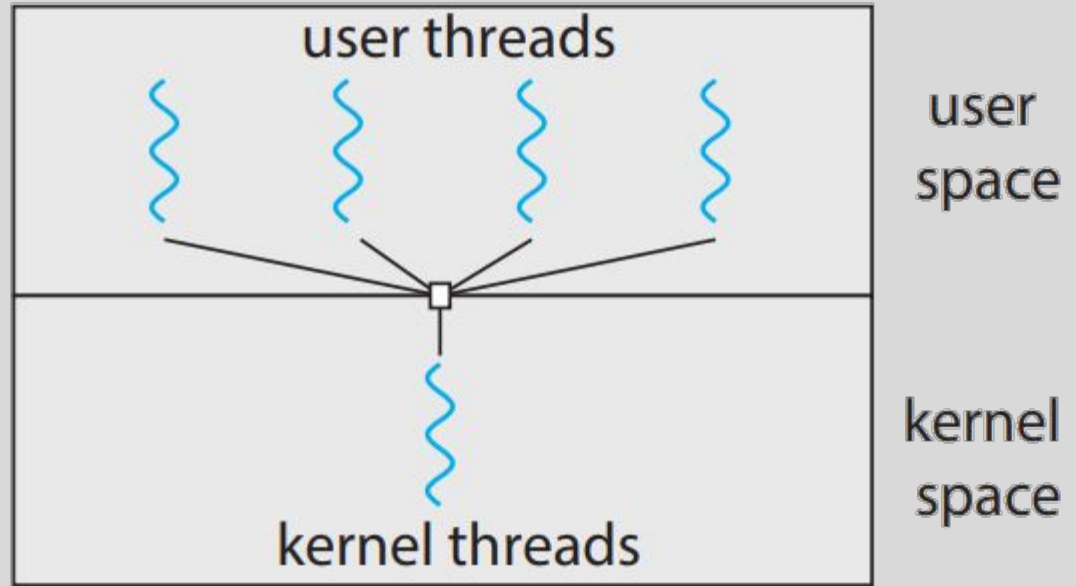
Modelo muchos a uno

Desventaja:

- Si el único hilo de kernel se bloquea, todo el proceso queda afectado.

Notar que no puede haber paralelismo real entre esos hilos en un sistema multinúcleo. El kernel solo ve una unidad de ejecución.

Este modelo conceptualmente es simple, pero presenta restricciones importantes en sistemas multicore.



Many-to-one model.

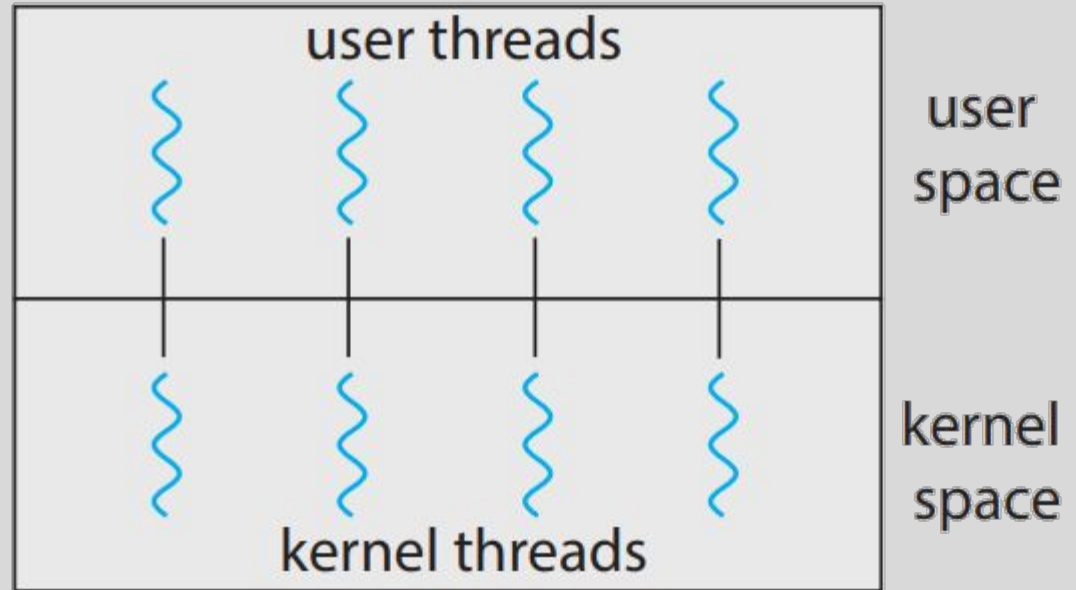
Modelo uno a uno

Cada hilo de usuario se asocia a un hilo de kernel.

Para cada hilo que crea la aplicación, existe una entidad correspondiente que el SO puede ver y planificar directamente.

El kernel no ve un solo flujo global, sino varios hilos independientes. Eso mejora la capacidad del sistema para:

- Seguir ejecutando otros hilos cuando uno se bloquea.
- Distribuir trabajo entre varios núcleos cuando el hardware lo permite.



One-to-one model.

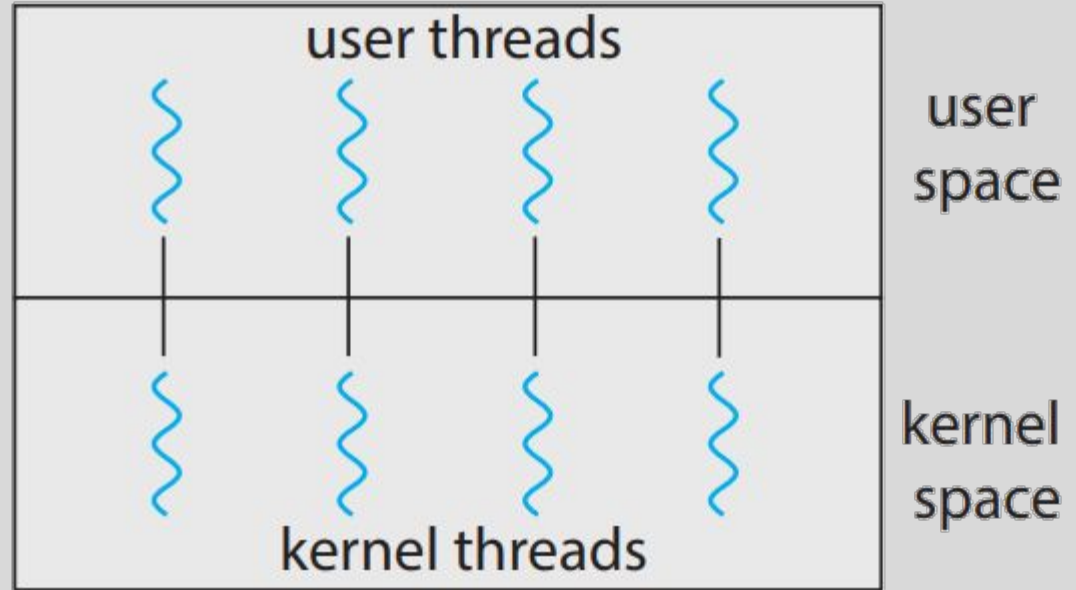
Modelo uno a uno

Ventajas

- Mejor concurrencia
- Un hilo bloqueado no frena los demás
- Permite paralelismo en multicore

Desventajas

- Más sobrecarga
- Demasiados hilos pueden degradar el rendimiento



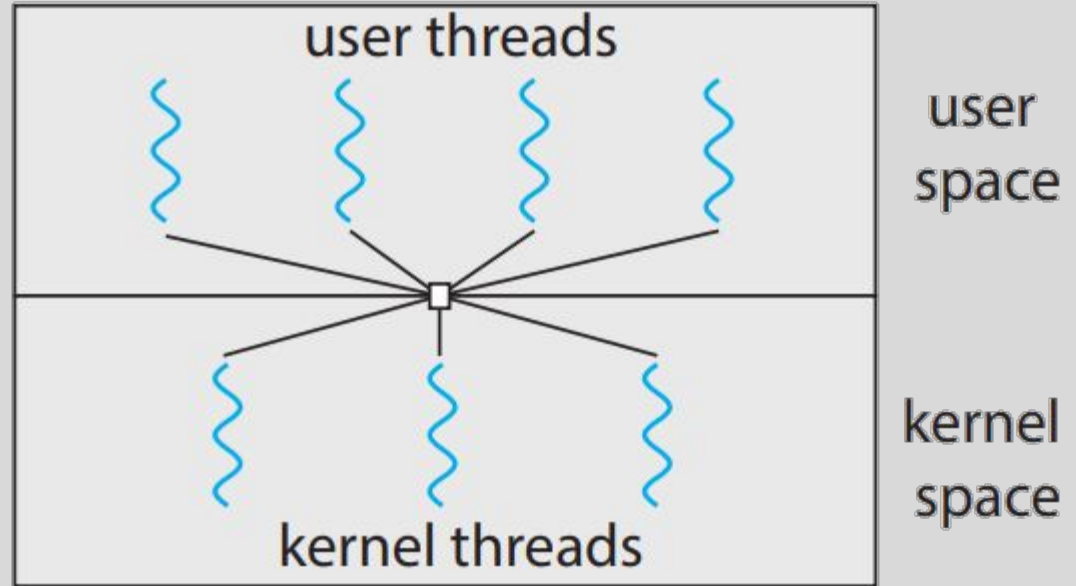
One-to-one model.

Modelo muchos a muchos

Múltiples hilos de usuario se **multiplexan** sobre un conjunto de hilos de kernel.

La aplicación puede crear varios hilos de usuario, pero no existe una correspondencia uno a uno entre un hilo de usuario y un hilo de kernel. La idea es combinar ventajas de los enfoques anteriores:

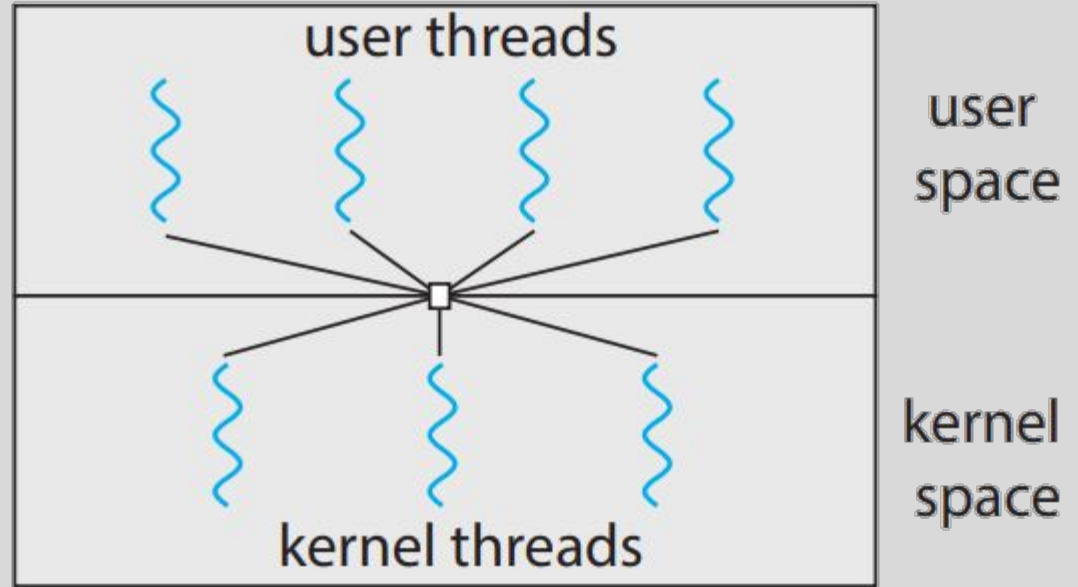
- Mantener flexibilidad a nivel de usuario
- No quedar atado a un único hilo de kernel
- Permitir que varios hilos puedan ejecutarse de forma paralela.



Many-to-many model.

Modelo muchos a muchos

Pero, su implementación es bastante más compleja, requiere coordinar cómo se asignan y reprograman los hilos de usuario sobre los hilos de kernel disponibles. Tarea para nada sencilla.



Many-to-many model.



Comparación de modelos

No hay un modelo mágicamente perfecto, cada uno prioriza algo distinto. Lo interesante es entender qué equilibrio logra cada uno entre:

- Costo
- Flexibilidad
- Bloqueo
- Visibilidad para el kernel
- Capacidad de aprovechar varios núcleos

Modelo	Idea general	Ventaja	Límite
Muchos a uno	varios user $\rightarrow$ 1 kernel	bajo costo	bloqueo y sin multicore
Uno a uno	1 user $\rightarrow$ 1 kernel	mejor concurrencia	más sobrecarga
Muchos a muchos	varios user $\rightarrow$ varios kernel	más flexible	más complejo



Qué modelo predomina hoy

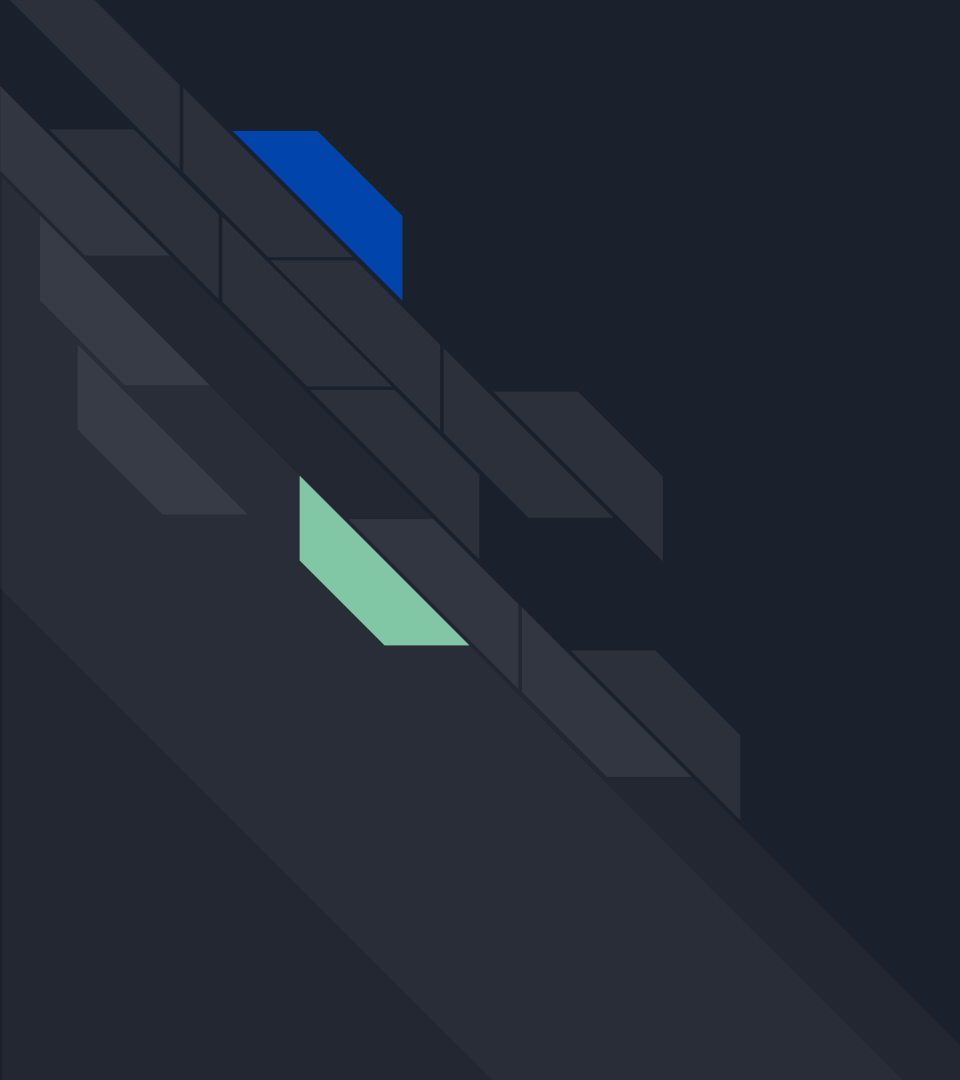
El modelo **muchos a muchos** parece conceptualmente muy atractivo, pero en la práctica su implementación resulta compleja.

Esto lleva a que muchos SO modernos adopten el modelo **uno a uno**. Porque:

- Es más directo
- El kernel puede planificar cada hilo explícitamente
- Maneja mejor bloqueos
- Y aprovecha de forma más natural el paralelismo de hardware

Hoy en día , en SO como Linux y Windows, domina el modelo **uno a uno**.

No significa que otras estrategias hayan desaparecido, pero sí que desde el punto de vista del SO de propósito general, el modelo uno a uno termina imponiéndose en la práctica.



LIBRERÍAS DE HILOS



Librerías de hilos: idea general

Los programadores no interactúan directamente con la estructura interna del SO. Lo hacen a través de librerías o APIs que permiten crear, administrar y coordinar hilos.

Una librería de hilos ofrece operaciones como:

- Crear o Finalizar un hilo
- Esperar su terminación
- En general, controlar su ciclo de vida.

Desde el punto de vista del programador, la librería proporciona una interfaz para poder expresar concurrencia dentro de un programa.

El programador trabaja sobre una API, no sobre el kernel directamente.



Pthreads, Windows y Java

Entre las librerías o entornos más clásicos y conocidos para trabajar con hilos aparecen:

- **Pthreads:** estándar POSIX ampliamente usado en sistemas UNIX y Linux.
- **Windows Threads:** soporte de hilos en sistemas Windows.
- **Java Threads:** modelo de hilos expuesto por la plataforma Java.

Si bien pertenecen a un ecosistema diferente, todos permiten expresar la misma idea general:

- Crear múltiples flujos de ejecución dentro de una aplicación.

No hace falta estudiar ahora su sintaxis particular.

Lo importante es entender que los hilos no son solo una idea teórica del SO, sino una herramienta concreta disponible en distintos entornos de programación.

PTHREADS

```
src > 03-threads > unix > C pthread_example_sum_of_n.c > ...
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int sum; /* this data is shared by the thread(s) */
6
7  void *runner(void *param); /* threads call this function */
8
9  int main(int argc, char *argv[])
10 {
11     pthread_t tid; /* the thread identifier */
12     pthread_attr_t attr; /* set of thread attributes */
13     /* set the default attributes of the thread */
14     pthread_attr_init(&attr);
15     /* create the thread */
16     pthread_create(&tid, &attr, runner, argv[1]);
17     /* wait for the thread to exit */
18     pthread_join(tid, NULL);
19     printf("sum = %d\n", sum);
20 }
21
22 /* The thread will execute in this function */
23 void *runner(void *param)
24 {
25     int i, upper = atoi(param);
26     sum = 0;
27     for (i = 1; i <= upper; i++)
28     {
29         sum += i;
30         // usleep(1000); // Sleep for 1 millisecond
31     }
32     pthread_exit(0);
33 }
34 // Compile with: gcc -o pthread_example_sum_of_n pthread_example_sum_of_n.c -lpthread
35 // Run with: ./pthread_example_sum_of_n 1000000
```


WINDOWS THREADS

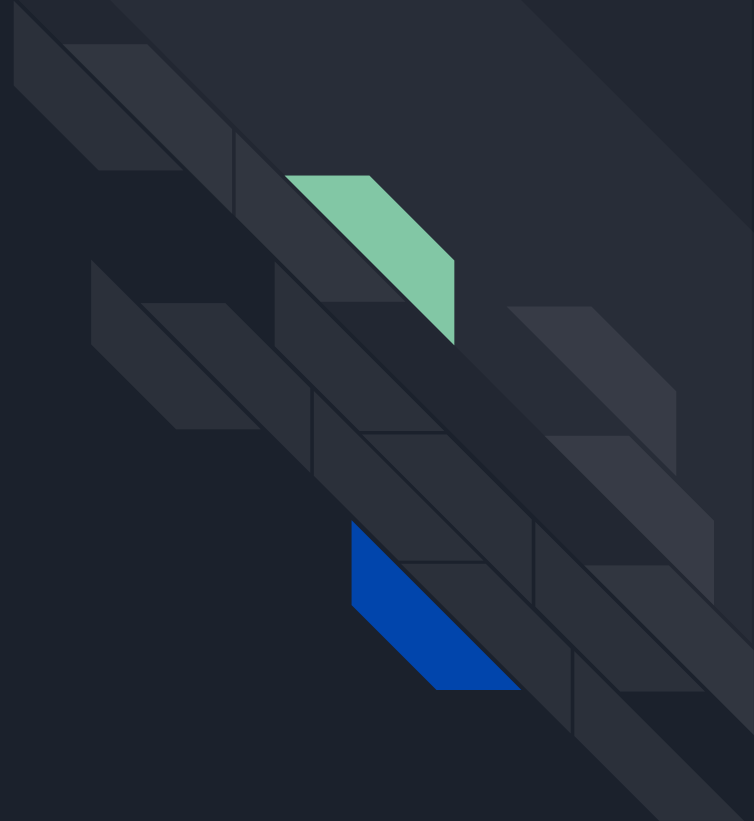
src > 03-threads > windows > C windows_sum_of_n.c > ...

```
1  #include <windows.h>
2  #include <stdio.h>
3
4  DWORD Sum; /* data is shared by the thread(s) */
5
6  /* The thread will execute in this function */
7  DWORD WINAPI Summation(LPVOID Param)
8  {
9      DWORD Upper = *(DWORD *)Param;
10     for (DWORD i = 1; i <= Upper; i++)
11         Sum += i;
12     return 0;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     DWORD ThreadId;
18     HANDLE ThreadHandle;
19     int Param;
20     Param = atoi(argv[1]);
21     /* create the thread */
22     ThreadHandle = CreateThread(
23         NULL,          /* default security attributes */
24         0,             /* default stack size */
25         Summation,      /* thread function */
26         &Param,         /* parameter to thread function */
27         0,             /* default creation flags */
28         &ThreadId); /* returns the thread identifier */
29     /* now wait for the thread to finish */
30     WaitForSingleObject(ThreadHandle, INFINITE);
31     /* close the thread handle */
32     CloseHandle(ThreadHandle);
33     printf("sum = %d\n", Sum);
34 }
```

JAVA

```
src > 03-threads > java > Driver.java
1  import java.util.concurrent.*;
2
3  class Summation implements Callable<Integer>
4  {
5      private int upper;
6      public Summation(int upper) {
7          this.upper = upper;
8      }
9
10     /* The thread will execute in this method */
11     public Integer call()
12     {
13         int sum = 0;
14         for (int i = 1; i <= upper; i++)
15             sum += i;
16         return sum;
17     }
18 }
19
20 public class Driver
21 {
22     public static void main(String[] args) {
23         int upper = Integer.parseInt(args[0]);
24         ExecutorService pool = Executors.newSingleThreadExecutor();
25         Future<Integer> result = pool.submit(new Summation(upper));
26         try {
27             System.out.println("sum = " + result.get());
28         }
29         catch (InterruptedException | ExecutionException ie) {
30             System.out.println("Exception: " + ie.getMessage());
31         }
32         finally {
33             pool.shutdown();
34         }
35     }
36 }
37 // The above code is a simple Java program that uses the Executor framework
```

OPERACIONES SOBRE HILOS





Crear hilos de forma asincrónica o sincrónica

De forma general, podemos pensar dos esquemas:

- **Creación asincrónica:** el hilo principal crea un hilo hijo y continúa su ejecución sin esperar a que termine.
- **Creación sincrónica:** el hilo principal crea uno o más hilos y luego espera a que finalicen para continuar.

Objetivo del primer caso: que distintas actividades avancen de manera concurrente e independiente.

En el segundo: dividir una tarea en partes, esperar los resultados parciales y luego combinarlos.



Cancelación de hilos: idea general

A veces un hilo debe terminar antes de completar su trabajo.

Esto puede ocurrir cuando una tarea ya no tiene sentido. Por ejemplo:

- Una operación interrumpida por el usuario
- El resultado de una operación dejó de ser necesario
- El programa decidió abortar una actividad en curso

La cancelación parece simple desde afuera, pero no siempre lo es.

Si un hilo está usando recursos o modificando datos compartidos, terminarlo de forma abrupta puede dejar al sistema en un estado inconsistente.



Cancelación asincrónica vs sincrónica

En términos generales, existen dos formas de cancelación:

- **Cancelación asincrónica:** un hilo termina inmediatamente a otro hilo.
- **Cancelación sincrónica:** el hilo objetivo verifica en ciertos puntos si debe finalizar, y se cancela de forma controlada.

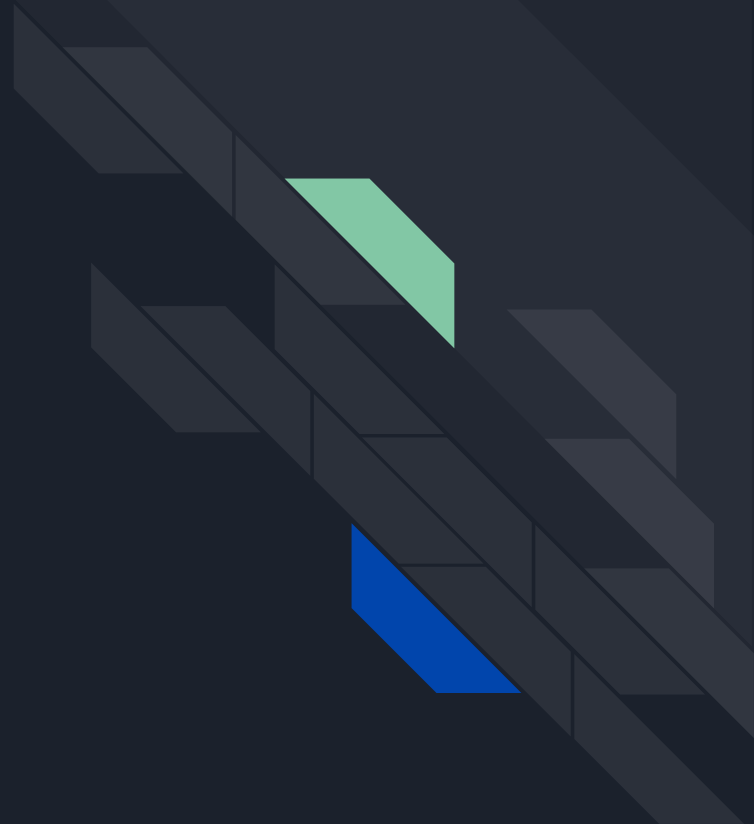
La diferencia es importante.

La cancelación asincrónica es riesgosa:

- El hilo podría estar usando recursos activamente o estar actualizando datos compartidos
- Dejando una operación a mitad de camino

La cancelación sincrónica permite que el hilo termine en un punto razonable, cuando puede liberar recursos o dejar sus datos en un estado consistente.

RESUMEN DE IDEAS





Resumen de ideas

- Proceso $\neq$ Hilo
- El hilo es la unidad de ejecución
- Los hilos comparten recursos del proceso
- Concurrencia $\neq$ Paralelismo
- Múltiples hilos aportan ventajas, pero también agregan complejidad
- Existen distintos modelos de implementación

Muchas Gracias

Jeremías Fassi

Javier E. Kinter

